

02 - How to implement a SWC to read/write DID in sender /receiver mode in FACEasy (Classic AutoSAR) ?

- Introduction
 - Sender/Receiver vs Client/Server modes.
 - Preconditions
- Step 1: Implement the SWC demo application
 - Step 1.a : RNIDL Files Creation
 - Step 1.b: SWC Code Generation and Modification
 - Step 1.c: SWC Code Compilation and Packaging
- Step 2: SWC Integration into device-cm-production
 - Preconditions
 - Step 2.a : Makefiles Updates
 - Step 2.b: SWC Connection Setup in RNIDL Files
 - Step 2.c: Final BSW Compilation
- Testing On PIU Board

Introduction

This tutorial aims to explain how to implement a SWC able to access a DID in **sender/receiver mode**, and integrate the new SWC into FACEasy (Classic AutoSAR).

Therefore it will be similar to the page [01 - How to implement a SWC to read/write DID in FACEasy \(Classic AutoSAR\) ?](#) which is focused on DID in **client /server mode**.

Sender/Receiver vs Client/Server modes.

As explained in the documentation <https://autosartutorials.com/common-terms-used-in-autosar/> (Ports And Ports Interfaces), there are 2 kinds of AUTOSAR Port Interfaces:

1. **ClientServerInterface**: it involves a client/server communication, allowing synchronous communication. The client triggers a callback executed on server side.
This is the focus of [01 - How to implement a SWC to read/write DID in FACEasy \(Classic AutoSAR\) ?](#) where an SWC is implementing runnables that are triggered when someone wants to read/write a DID managed by the SWC.
2. **SenderReceiverInterface**: this is based on simple asynchronous data exchange between entities.
This tutorial will focus on this part, and explain how to develop a simple SWC that will run and modify the value of a specific DID. Then, by using DDT2000 it will be possible to check the value updated by the SWC.

Preconditions

Access to the 2 projects that we will update during this tutorial is necessary:

- diagnostic-management : <https://gitlabee.dt.renault.com/swlabs/face/ez1/sw-framework/diagnostic-management>
- device-cm-production : <https://gitlabee.dt.renault.com/swlabs/face/ez1/cp/device-cm-production>

First of all, we have to use a DID supported by the system, that is to say we wither have to use an already existing DID, or create a new one.

This is achieved in the **Diag_RN_Candela_v2.3.xlsx** document located in device-cm-production/sysdesc/device/inputs/diagextract/CanDelaStudio/ (ex: https://gitlabee.dt.renault.com/swlabs/face/ez1/cp/device-cm-production/-/blob/master/sysdesc/device/inputs/diagextract/Diag_RN_Candela_v2.3.xlsx).

- To add a new DID, update the sheet "DataIdentifier" and provide the various details about the desired new DID (name, read/write rights, size, units, etc...).
- This Excel fil should then be processed by CandelaStudio to generate a **DiagExtract.arxml** file. (TODO: explain the process here)
- Keep the new DiagExtract.arxml, we will use it later.
- To reuse an existing DID, no update to the Excel document is required.

Here is a screenshot of a typical "DataIdentifier" sheet of this document:

DiagnosticDataByIdentifier				dataIdentifier		dataElement		dataElement				compuMethod	
longName	Name	ReadData	WriteData	id	didSize	bitOffset	shortName	maxNumberOfElemen	unit	compu	con		
DidTestNm_16bytes	DidTestNm_16bytes	x	x	0xF230	16	0	TestNm_16bytes	16	NoUnit				
DidTestNm_4bytes	DidTestNm_4bytes	x	x	0xF231	4	0	TestNm_4bytes	4	NoUnit				
DidTestDiag_16bytes	DidTestDiag_16bytes	x	x	0xF232	16	0	TestDiag_16bytes	16	NoUnit				
DidTestDiag_4bytes	DidTestDiag_4bytes	x	x	0xF233	4	0	TestDiag_4bytes	4	NoUnit				
F012_OperationalReference	F012_OperationalReference	x		0xF012	10	0	F012_OperationalReference	10	NoUnit				
F187_VehicleManufacturerSparePartNumber_R	F187_VehicleManufacturer	x		0xF187	10	0	F187_VehicleManufacturerSparePartNumber_R	10	NoUnit				
F188_ConfigurationFileReferenceLinkRUC	F188_ConfigurationFileRef	x		0xF188	10	0	F188_ConfigurationFileReferenceLinkRUC	10	NoUnit				
F18A_SystemSupplierNumber	F18A_SystemSupplierNum	x		0xF18A	10	0	F18A_SystemSupplierNumber	10	NoUnit				
F18C_EcuSerialNumberDataIdentifier	F18C_EcuSerialNumberData	x		0xF18C	20	0	F18C_EcuSerialNumberDataIdentifier	20	NoUnit				
F190_VIN	F190_VIN	x	x	0xF190	17	0	F190_VIN	17	NoUnit				
F191_VehicleManufacturerECUHardwareNumber	F191_VehicleManufacturer	x		0xF191	10	0	F191_VehicleManufacturerECUHardwareNumber	10	NoUnit				
F194_SystemSupplierECUSoftwareNumber	F194_SystemSupplierECU	x		0xF194	10	0	F194_SystemSupplierECUSoftwareNumber	10	NoUnit				
F195_SystemSupplierECUSoftwareVersionNumber	F195_SystemSupplierECU	x		0xF195	32	0	F195_SystemSupplierECUSoftwareVersionNumber	32	NoUnit				
F1A0_VDIAG	F1A0_VDIAG	x		0xF1A0	1	0	F1A0_VDIAG	1	NoUnit				

In this tutorial, we will be using the DID: [DidTestDiag_16bytes](#)



Compatibility between access modes

Warning: the access mode (client/server or sender/receiver) is not part of the DID description at this level. That is to say, any DID can be accessed in client/server or in sender/receiver mode.

But, for a DID accessed by multiple SWCs in different modes, the dedicated ports that will be created in the Autosar middleware will be incompatible with each-other.

So, it is better to stick to one access type per DID.

Step 1: Implement the SWC demo application

This chapter explains how to create a new demo SWC to test the DID reading/writing.

In the diagnostic-management project:

- create a new git branch, ex. `FACESWP-4542_demo_did_sender_receiver`
- create a new folder to host the new SWC: `demo/swcReadWriteDidSenderReceiver`



Note: the implementation detailed in this chapter can be found here : <https://gitlabee.dt.renault.com/swlabs/face/ez1/sw-framework/diagnostic-management/-/commit/41373f80e7804fb3a78af3278c1bab85737eb85e>

Step 1.a : RNIDL Files Creation

In this step, we will modify/create the following files:

- `demo/CMakeLists.txt`
- `demo/swcReadWriteDidSenderReceiver/itdDid.rnil`
- `demo/swcReadWriteDidSenderReceiver/swcReadWriteDidSenderReceiver.rnidl`
- `demo/swcReadWriteDidSenderReceiver/CMakeLists.txt`

Update the demo `CMakeLists.txt` to point to the new SWC:

diagnostic-management/demo/CMakeLists.txt

```
...  
add_subdirectory(swcReadWriteDidSenderReceiver)
```

Create the file `demo/swcReadWriteDidSenderReceiver/itdDid.rnidl` (or choose another filename if you prefer):

- Create an interface of type `senderReceiver` ; its name should follow the convention **`DataServices_{Data}`**.
In our case, the name of the interface should be **`DataServices_DidTestDiag_16bytes`**.

demo/swcReadWriteDidSenderReceiver/itfDid.rnidl

```
package face.swcReadWriteDidSenderReceiver  
  
import "itf_AutosarDcm.arxml"  
  
// DidTestDiag_16bytes  
typedef UInt8[16] DidTestType  
interface senderReceiver serviceBsw DidTestType DataServices_DidTestDiag_16bytes
```



TODO:

- explain serviceBsw
- explain what is the contents of the interface exactly

Create a new file `demo/swcReadWriteDidSenderReceiver/swcReadWriteDidSenderReceiver.rnidl`.

This is the main RNIDL file describing our component : its contents shall follow RNIDL guidelines and use AUTOSAR abstractions, which will later allow the generation of a C code skeleton for our SWC implementation.

Note: this file should have the same name as the swc folder name (`swcReadWriteDidSenderReceiver`).

The aim of the SWC will be to run a process scheduled each 100ms, to write new values in the DID and read back the values to check that it was correctly written. The read/write operations will follow the sender/receiver mechanism, that is to say they will not be triggered by external requests but simply happen @10 Hz.

AUTOSAR provides variable abstractions to allow access to memory location in an ECU and transferring its contents to the measurement & calibration system:

- For **Client/Server**, variable abstraction is retrieved in the **ArgumentDataPrototype** of a **ClientServerOperation** of a **ClientServerInterface**.
- For **Sender/Receiver**: variable abstraction is accessed using a **VariableDataPrototype** of a **SenderReceiverInterface** used in a **PortPrototype**. The vdp is used to handle data in a non volatile memory block, and represents an atomic piece of information transmitted among PortPrototypes.



Figure 4.1: **dataElements** of a **SenderReceiverInterface**

Extract from the official Autosar documentation (AUTOSAR_TPS_SoftwareComponentTemplate.pdf)

So here, to access the DID value, the `read()` and `write()` operations will rely on Variable Data Prototypes (vsp), which are local copies of the data (1 vdp per operation). It is not possible to use only 1 read & write port (rport) because this is only supported for `NvDataInterface`.

In the rnidl, we specify:

- 2 ports for the communication:
 - rport : requested port used by the SWC to read data from the outside
Naming must follow: **rport DataServices_{Data} RP_{PortName}**
Example: **rport DataServices_DidTestDiag_16bytes RP_DidTest**
 - pport : provided port used by the SWC to write data to the outside
Naming must follow: **pport DataServices_{Data} PP_{PortName}**
Example: **pport DataServices_DidTestDiag_16bytes PP_DidTest**
- 2 servicesNeeds, 1 per port:
 - One used for reading the data. It will use a vdp (VariableDataPrototype) that is provided by the requested port
Naming: **vdp signalBasedDiagnostics RP_{PortName}.Data**
Example: **vdp signalBasedDiagnostics RP_DidTest.Data**
 - The other one is for writing the data.
Naming: **vdp signalBasedDiagnostics PP_{PortName}.Data**
Example: **vdp signalBasedDiagnostics PP_DidTest.Data**
- 1 runnable called **rTestReadWriteDidSenderReceiver** that will be able to both read and write the data.
 - access the 2 data provided/requested by the ports:
dataAccess explicit RP_DidTest.Data
dataAccess explicit PP_DidTest.Data

demo/swcReadWriteDidSenderReceiver/swcReadWriteDidSenderReceiver.rnidl

```
package face

import "itfDid.rnidl"

// SWC ID = RWD
swcAppl swcReadWriteDidSenderReceiver {
    version {major 0 minor 1}

    property (multiInstance = false)
    rport DataServices_DidTestDiag_16bytes RP_DidTest
    pport DataServices_DidTestDiag_16bytes PP_DidTest

    // This runnable will result in C code generation
    runnable rTestReadWriteDidSenderReceiver {
        event period 100
        dataAccess explicit PP_DidTest.Data
        dataAccess explicit RP_DidTest.Data
    }

    // serviceNeeds for reading side
    serviceNeeds diagValue Did_Read {
        vdp signalBasedDiagnostics RP_DidTest.Data
        property (read)
    }

    // serviceNeeds for writing side
    serviceNeeds diagValue Did_Write {
        vdp signalBasedDiagnostics PP_DidTest.Data
        property (write)
    }
}
```



Open questions:

1. What is diagValue ? is it "DiagnosticValueNeeds" child class of ServiceNeeds ?
2. What is signalBasedDiagnostics ?
signalBasedDiagnostics indicates the Role BasedDataAssignment shall be used for signal based diagnostics. With this assignment, the role of the data can be mapped to a specific ServiceNeeds element, so that a tool is able to create the correct access.
3. is the instance of the serviceNeeds important ? (ex: DidTest_Read : can we use any name here?)
4. What is RP_DidTest.Data : there is no Data in RP_DidTest ...

Add the CMakeLists.txt for the new SWC:

demo/swcReadWriteDidSenderReceiver/CMakeLists.txt

```
# import build scripts
find_package(BuildScripts REQUIRED CONFIG)

# import common ARXML
find_package(CommonArxml REQUIRED CONFIG)

include(AddCpGenerate)
include(AddCpLibrary)

set(SWCNAME swcReadWriteDidSenderReceiver)
set(CP_PACKAGENAME ${SWCNAME})

# Generate cp arxml for a swc based on rnidl/fidl
add_cp_generate(${CP_PACKAGENAME}
    ${SWCNAME}.rnidl
    SWCNAME ${SWCNAME}
    IMPORTS
        # import basic elements (types, units and swaddr methods)
        ${COMMON_ARXML_DIR}/type
        ${COMMON_ARXML_DIR}/unit
        ${COMMON_ARXML_DIR}/swaddr

    INCLUDES
        ${COMMON_ARXML_DIR}/itf
        ${CONAN_INTERFACESARXML_ROOT}/lib/fidl
)

# Create a library which includes the source files
# add_cp_library(${CP_PACKAGENAME} src/swcReadWriteDidSenderReceiver.c EXPORTEDPATHS src RTE)
```



Note: the last line in this CMakeLists.txt is commented, as the .c file does not exist yet.

Step 1.b: SWC Code Generation and Modification

Generate the C code using the following commands:

Generate ARXML and C files

```
cd ~/diagnostic-management

. setenv.sh # set the environment
clean .     # remove the build directory if exists
install .   # fetch and install dependencies
build .     # compile rnidl files, generate arxml files, C template code and RTE header files
```

The following files should have been generated:

Generated Files

```
build/demo/swcReadWriteDidSenderReceiver/swcReadWriteDidSenderReceiver/testenv/rte/
    Rte_swcReadWriteDidSenderReceiver.h
    Rte_swcReadWriteDidSenderReceiver_Type.h
    Rte_Type.h
    swcReadWriteDidSenderReceiver.c
    swcReadWriteDidSenderReceiver_MemMap.h
```

In particular, it contains a basic implementation of the runnable `rTestReadWriteDidSenderReceiver` which we are going to update. Copy this file locally in the `src/` folder:

```
cp build/demo/swcReadWriteDidSenderReceiver/src/swcReadWriteDidSenderReceiver.c demo
/swcReadWriteDidSenderReceiver/src/
```

In this file, let's update the function `rTestReadWriteDidSenderReceiver` and implement the behavior needed:

- First, read the DID value using the **Rte_Read_RP_DidTest_data()** operation generated by AUTOSAR tools
- Then, write the DID value using the **Rte_Write_PP_DidTest_data()** operation generated by AUTOSAR tools:
 - if the 1st byte of the DID value read is 0xAB, then we overwrite the value with 0xCD
 - else if the 1st byte of the DID value read is 0x01, then we overwrite the value with 0xEFEFEFEFEFEFEFEFEFEFEFEFEFEFEFEFEFEF
 - otherwise do not update the value

swcReadWriteDidSenderReceiver.c Updates

```

FUNC(void, RTE_CODE) rTestReadWriteDidSenderReceiver (void)
{
    // Read the DID value
    DidTestType arg_Read_RP_DidTest_data_data;
    Std_ReturnType /*@alt void@*/ ret_Read_RP_DidTest_data;
    ret_Read_RP_DidTest_data = Rte_Read_RP_DidTest_data (arg_Read_RP_DidTest_data_data);

    // Check that the read operation was successful
    Std_ReturnType /*@alt void@*/ ret_final;
    if (ret_Read_RP_DidTest_data != RTE_E_OK) {
        ret_final = RTE_E_INVALID;
    } else {
        // Check the 1st byte of the DID value and overwrite the data
        // following this logic:
        // - 0xAB -> Fill the data with 0xCD
        // - 0x01 -> Fill the data with 0xEF
        // - otherwise do not update the DID value

        DidTestType arg_Write_PP_DidTest_data_data;
        Std_ReturnType /*@alt void@*/ ret_Write_PP_DidTest_data;

        // Case 0xAB
        if (arg_Read_RP_DidTest_data_data[0] == 0xAB) {
            // Set the data and write them
            for (int i = 0; i < 16; i++) {
                arg_Write_PP_DidTest_data_data[i] = 0xCD;
            }
            ret_Write_PP_DidTest_data = Rte_Write_PP_DidTest_data (arg_Write_PP_DidTest_data_data);
            (void)ret_Write_PP_DidTest_data;

            // Case 0x01
        } else if (arg_Read_RP_DidTest_data_data[0] == 0x01) {
            // Set the data and write them
            for (int i = 0; i < 16; i++) {
                arg_Write_PP_DidTest_data_data[i] = 0xEF;
            }
            ret_Write_PP_DidTest_data = Rte_Write_PP_DidTest_data (arg_Write_PP_DidTest_data_data);
            (void)ret_Write_PP_DidTest_data;
        }

        // Otherwise, no update of the DID value
    }
    (void)ret_final;
}

```

 TODO: what to do with the final result ? There is no possibility to return something ?

Step 1.c: SWC Code Compilation and Packaging

Update the `CMakeLists.txt` to take into account the new C file:

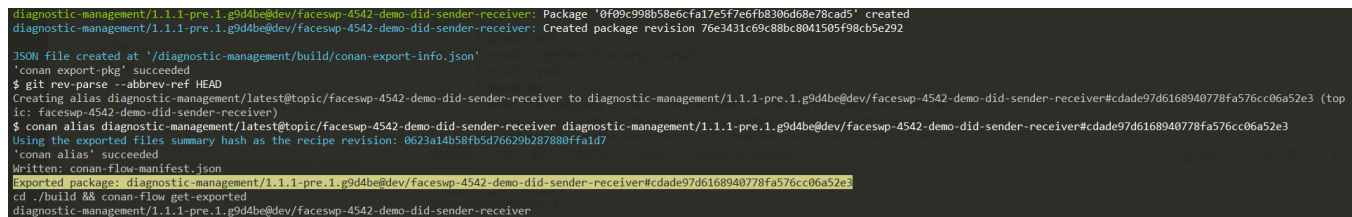
demo/swcReadWriteDidSenderReceiver/CMakeLists.txt

```
...
# Create a library which includes the source files
add_cp_library(${CP_PACKAGE_NAME} src/swcReadWriteDidSenderReceiver.c EXPORTEDPATHS src RTE)
```

Finally, compile and package the SWC:

```
build . # compile rnidl files, generate arxml files, C template code and RTE header files
package . # generate a local package
```

Here is a screenshot of a typical result of a SWC package generation:



```
diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver: Package '0f09c998b58e6cf17e5f7e6fb8396d68e78cad5' created
diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver: Created package revision 76c3431c69c88bc8041505f98cb5e292

JSON file created at '/diagnostic-management/build/conan-export-info.json'
'conan export-pkg' succeeded
$ git rev-parse --abbrev-ref HEAD
Creating alias diagnostic-management/latest@topic/faceswp-4542-demo-did-sender-receiver to diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver#cdade97d6168940778fa576cc06a52e3 (top
ic: faceswp-4542-demo-did-sender-receiver)
$ conan alias diagnostic-management/latest@topic/faceswp-4542-demo-did-sender-receiver diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver#cdade97d6168940778fa576cc06a52e3
Using the exported files summary hash as the recipe revision: 0623a14b58f05d76629b287880ffaid7
'conan alias' succeeded
Written: conan-flow-manifest.json
Exported package: diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver#cdade97d6168940778fa576cc06a52e3
cd ./build && conan-flow get-exported
diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver
```

Normally, the SWC should be compiled as a static lib: `./build/demo/swcReadWriteDidSenderReceiver`
`/libswcReadWriteDidSenderReceiver.a`

Step 2: SWC Integration into device-cm-production

This chapter explains how to instantiate our new SWC and connect it to the rest of the AutoSAR system.

This takes place in the project **device-cm-production**, where you should create a new git branch, ex. `FACESWP-4542_demo_did_sender_receiver`.

 Note: the implementation detailed in this chapter can be found here : <https://gitlabee.dt.renault.com/swlabs/face/ez1/cp/device-cm-production/-/commit/5979efedfb7f746c895e46235b76bb0458f5fec7>

Preconditions

If a new DID has been added in CandelaStudio (i.e. new DID was added to **Diag_RN_Candela_v2.3.xlsx**), update **DiagExtract.arxml** in the following folder:

`device-cm-production/sysdesc/device/inputs/diagextract/CanDelaStudio/`

If using a DID already existing in CandelaStudio, **DiagExtrace.arxml** should not need any update and should already be correct.

Step 2.a : Makefiles Updates

First, we have to update the Conan configuration to add a new dependency to our new SWC, in `sysdesc/conandata.yml` :

sysdesc/condandata.yml

```
- ...
- diagnostic-management/1.1.0@face/stable
- chassis/1.0.3@face/stable
- diagnostic-management/latest@topic/faceswp-4542-demo-did-sender-receiver # <-- New dependency
```

Here you can either set:

- Use the local branch name that was issued by the 'package .' command in diagnostic-management:
diagnostic-management/1.1.1-pre.1.g9d4be@dev/faceswp-4542-demo-did-sender-receiver
- Use a global branch name in case you have already pushed / published your branch:
diagnostic-management/latest@topic/faceswp-4542-demo-did-sender-receiver

Then, we can reference the SWC `swcReadWriteDidSenderReceiver` in the makefile `device-cm-production/sysdesc/device/CMakeLists.txt`:

sysdesc/device/CMakeLists.txt

```
...
add_cp_ecu(${PROJECT_NAME} ${PROJECT_NAME}.rnidl "bswconfig/CpEcuc_rteEvtMap.rnidl"
  SWCPACKAGES
    ...
    # packages from diagnostic-management
    "swcReadWriteDid"
    "swcReadWriteDidSenderReceiver" # <-- new SWC
    ...
```

Step 2.b: SWC Connection Setup in RNIDL Files

The file `sysdesc/device/inputs/piuCabinMaster.rnidl` imports several files that we are going to update:

- `mapswcPiuCabinMaster.rnidl`
- `mapsigPiuCabinMaster.rnidl`
- `mapdiagPiuCabinMaster.rnidl`
- `bswconfig/CpEcuc_rteEvtMap.rnidl`

Instantiate the SWC in the Cabin Master, in the file `device-cm-production/sysdesc/device/swcCabinMaster.rnidl`:

sysdesc/device/swcCabinMaster.rnidl

```
...
// Test Did Sender Receiver
swc swcReadWriteDidSenderReceiver swcReadWriteDidSenderReceiver
```

Then, map the DID to the vdp defined in our SWC : update the file `device-cm-production/sysdesc/device/mapdiagPiuCabinMaster.rnidl`

Here we have 2 vdp named **RP_DidTest.Data** and **PP_DidTest.Data**, which needs to be connected to [DidTestDiag_16bytes](#).

The syntax is the following: **vsp {SWC}.RP_{PortName}.Data {DataName}.{ParamName}**

Example: **vdp swcReadWriteDidSenderReceiver.RP_DidTest.Data DidTestDiag_16bytes.TestDiag_16bytes**


```

...
// swcReadWriteDidSenderReceiver
vdp swcReadWriteDidSenderReceiver.PP_DidTest.Data DidTestDiag_16bytes.TestDiag_16bytes
vdp swcReadWriteDidSenderReceiver.RP_DidTest.Data DidTestDiag_16bytes.TestDiag_16bytes

```

DiagExtract.arxml

```
<DIAGNOSTIC-DATA-IDENTIFIER>  
  <SHORT-NAME>DidTestDiag_16bytes</SHORT-NAME>                                <!-- DID Name -->  
  <ID>62002</ID>  
  <DATA-ELEMENTS>  
    <DIAGNOSTIC-PARAMETER>  
      <BIT-OFFSET>0</BIT-OFFSET>  
      <DATA-ELEMENTS>  
        <DIAGNOSTIC-DATA-ELEMENT>  
          <SHORT-NAME>TestDiag_16bytes</SHORT-NAME>      <!-- Param Name -->  
          <ARRAY-SIZE-SEMANTICS>FIXED-SIZE</ARRAY-SIZE-SEMANTICS>  
          <MAX-NUMBER-OF-ELEMENTS>16</MAX-NUMBER-OF-ELEMENTS>  
          <SW-DATA-DEF-PROPS>  
            <SW-DATA-DEF-PROPS-VARIANTS>  
              <SW-DATA-DEF-PROPS-CONDITIONAL>  
                <BASE-TYPE-REF DEST="SW-BASE-TYPE"/>Diagnostics  
  
                <COMPU-METHOD-REF DEST="COMPU-METHOD">  
  
                  /Common/BaseTypes/uint8_opaq</BASE-TYPE-REF>  
  
                  <COMPU-METHOD-REF DEST="COMPU-METHOD">  
  
                    /Diagnostics/Common/CompuMethods/Identical_001</COMPU-METHOD-REF>  
  
                    </SW-DATA-DEF-PROPS-CONDITIONAL>  
                  </SW-DATA-DEF-PROPS-VARIANTS>  
                </SW-DATA-DEF-PROPS>  
              </DIAGNOSTIC-DATA-ELEMENT>  
            </DATA-ELEMENTS>  
          </DIAGNOSTIC-PARAMETER>  
        </DATA-ELEMENTS>  
      <DID-SIZE>16</DID-SIZE>  
    </DIAGNOSTIC-DATA-IDENTIFIER>
```

sysdesc/device/bswconfig/CpEcuc_rteEvtMap.rnidl

```
// --- 100 ms ---
...
eventMap swcReadWriteDidSenderReceiver evt_100ms_rTestReadWriteDidSenderReceiver
```

Build the sysdesc and PIU images with the following commands:

Sysdesc and BSW generation

```
cd ~/device-cm-production

clean sysdesc
install sysdesc # Check that you get a similar message in the logs: diagnostic-management/1.1.1-pre.1.
g9d4be@dev/faceswp-4542-demo-did-sender-receiver: Already installed!
build sysdesc   # If this step fails, check that conandata.yml has the correct dependency on the new SWC

clean bsw
install bsw
build bsw
```

Testing On PIU Board

Once the SWC has been built and flashed on the PIU, we can check with the behavior DDT2000, and confirm that it is what we expected:

	Command	Answer	Description
Check Default DID Value	22F232	62F232000000000000000000000000 000000000000	Request DID F232 Value Reply shows that DID Value is 000000000000000000000000000000 000
Set DID to a random value	2EF23212312312312312312312312312312312312	6EF232	Set the DID Value to 12312312312312312312312312312312312 The reply shows that the command was successful
	22F232	62F23212312312312312312312312312312312312 12312312312	Request DID F232 Value Reply shows that DID Value is 12312312312312312312312312312312 312 This value was <u>not changed by our SWC</u>
Set the DID to 0xAB*	2EF232AB000000000000000000000000 000000000000	6EF232	Set the DID Value to AB0000000000000000000000000000000000 The reply shows that the command was successful
	22F232	62F232CDCDCDCDCDCDCDCDCDCDCDCDCDCDCDCD DCDCDCDCDCD	Request DID F232 Value Reply shows that <u>our SWC updated the DID Value to CDCDCDCDCDCD</u> CDCDCDCDCDCDCDCDCDCDCD
Set the DID to 0x01*	2EF23201000000000000000000000000 000000000000	6EF232	Set the DID Value to 010000000000000000000000000000000000 The reply shows that the command was successful
	22F232	62F232EFEFEFEFEFEFEFEFEFEFEFEFEFEFEFE FEFEFEFEFEF	Request DID F232 Value Reply shows that <u>our SWC updated the DID Value to EFEFEFEFEFEF</u> EFEFEFEFEFEFEFEFEFEF